สมาชิกและหน้าที่ที่รับผิดชอบ

นายธนดล ไชยศิลา	673380585-0	Network Architects
นางสาวกัญญาภัค ทองวิเศษ	673380391-3	Network Architects
นางสาวนันทพร ลุนทอง	673380409-0	IaC/DevOps
นางสาวอลิชา ชนะบุญ	673380431-7	IaC/DevOps
นางสาวปรายฝน ฮกเซ็ง	673380591-5	QA/Test
นางสาวปวริศา สีดาชมภู	673380592-3	QA/Test

Lab 4 using your specific repository, layout, and networking assumptions. This version **treats Local subnets as private networks** and **treats the remote server / inter-site network as a 10.10.x.x "internet/public network"** for the purposes of NAT and routing logic. It also assumes the repository hierarchy:

```
None
automation/
├── mockup-infra/
└── week02-stateless-stateful/
    └── phase1-mockup/
```

We will produce:

✅ A revised **high-fidelity lab plan** ✅ **Detailed addressing & topologies** ✅ **Cisco-themed network section** ✅ **IaC / VS Code section** ✅ **Troubleshooting and engineering reflection** ✅ A complete **README.md** ready to be committed to the repo

---

🧪 LAB 4 Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

---

🎯 LAB GOALS

1. **Network Setup:** Create two local labs (A & B) connected via a simulated "public/internet" network (10.10.0.0/16)
2. **NAT + Routing:** Configure static routes and NAT so local traffic reaches remote services
3. **Application Deployment:** Use `mockup-infra` from your repository to deploy stateless and stateful APIs
4. **Testing:** Compare how sessions behave locally vs across the simulated internet

---

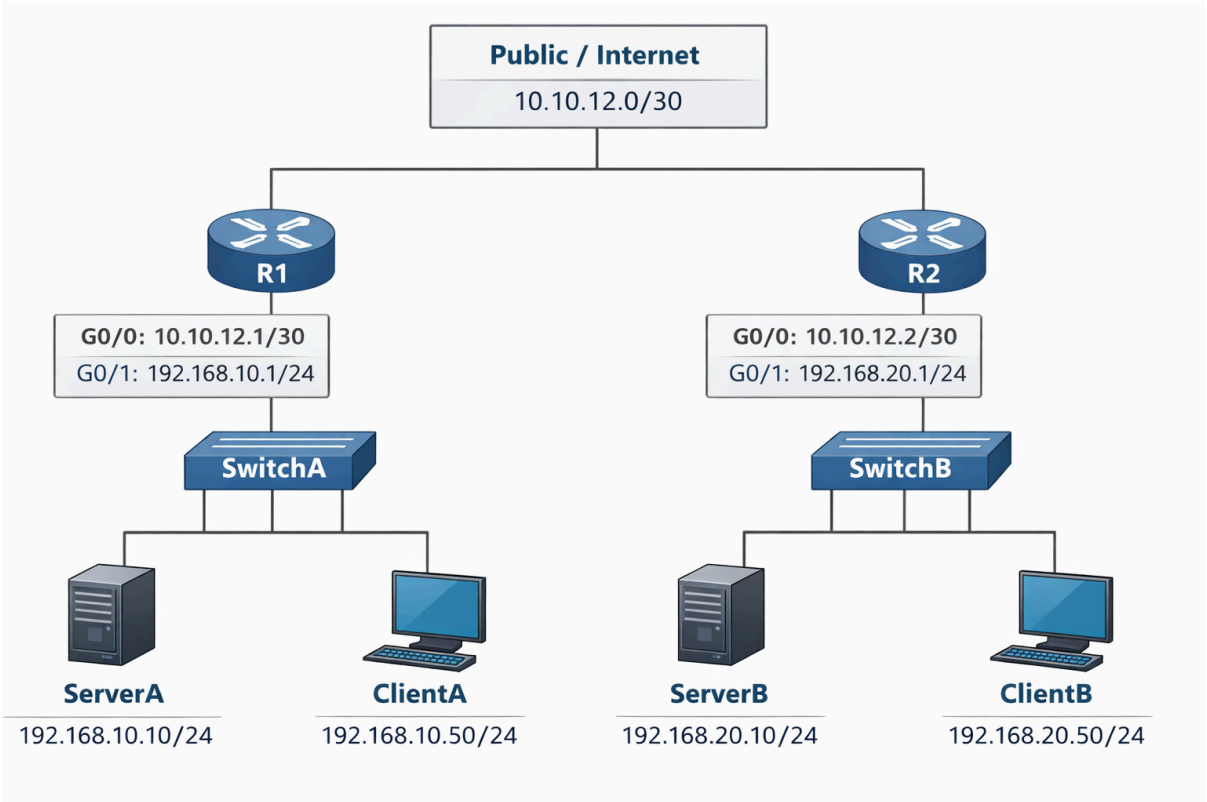
👥 TEAM BREAKDOWN (5–6 students)

Role	Count	Focus
Network Architects	2	Router configs, Cisco CLI
IaC/DevOps	2	VS Code, mockup-infra
QA/Test	1–2	Server/Client scripting, session behavior

🗺️ UPDATED NETWORK TOPOLOGY

```python
                10.10.12.0/30 "Public/Internet"
             ┌──────────────────────────┐
             |                          |
         R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
             |                          |
   Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
             |                          |
         SwitchA                     SwitchB
             |                          |
 ServerA (mockup)            ServerB (mockup)
 ClientA                         ClientB
```

We treat 10.10.12.0/30 as a **simulated internet** for the purposes of NAT + public reachability.



📍 ADDRESS TABLE

Component	Interface	IP Address	Subnet Mask	Default Gateway	Role
R1	G0/0	10.10.12.1	255.255.255.252	-	Public
R1	G0/1	192.168.10.1	255.255.255.0	-	LAN A Gateway
ServerA	eth0	192.168.10.10	255.255.255.0	192.168.10.1	Mockup infra A
ClientA	eth0	192.168.10.50	255.255.255.0	192.168.10.1	Tester
R2	G0/0	10.10.12.2	255.255.255.252	-	Public
R2	G0/1	192.168.20.1	255.255.255.0	-	LAN B Gateway

Component	Interface	IP Address	Subnet Mask	Default Gateway	Role
ServerB	eth0	192.168.20.10	255.255.255.0	192.168.20.1	Mockup infra B
ClientB	eth0	192.168.20.50	/24		Tester

🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

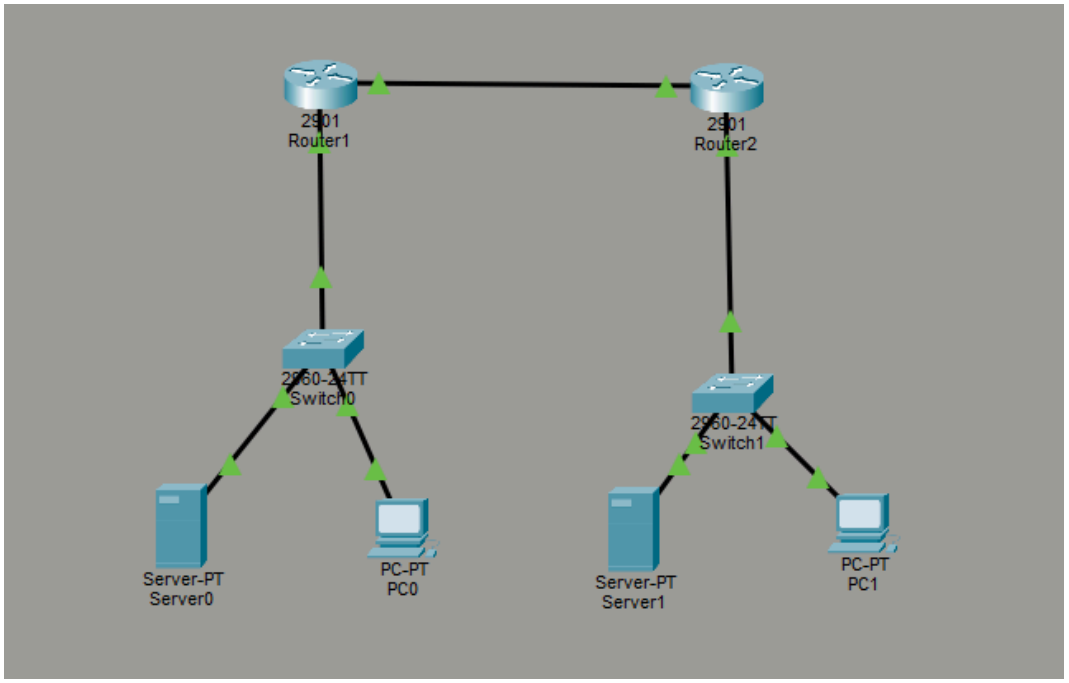
🧱 R1 CONFIGURATION (LAN A)

```
None
enable
conf t

hostname R1

interface g0/0
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

🧱 R2 CONFIGURATION (LAN B)

```
None
enable
conf t

hostname R2

interface g0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.10.0 255.255.255.0 10.10.12.1

access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

หลังจาก Config R1 & R2



🧪 ROUTING & NAT VERIFICATION

On R1 & R2:

```
None
show ip route
show ip nat translations
```

R1 :

```
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0
L       10.10.12.1/32 is directly connected, GigabitEthernet0/0
     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/1
L       192.168.10.1/32 is directly connected, GigabitEthernet0/1
S    192.168.20.0/24 [1/0] via 10.10.12.2

R1#show ip nat translations
R1#
```

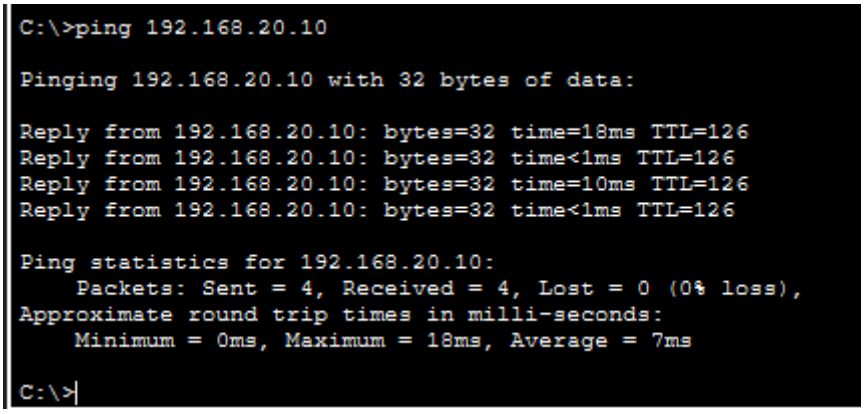
R2 :

```
R2#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is 10.10.12.1 to network 0.0.0.0

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0
L       10.10.12.2/32 is directly connected, GigabitEthernet0/0
S    192.168.10.0/24 [1/0] via 10.10.12.1
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/1
L       192.168.20.1/32 is directly connected, GigabitEthernet0/1
S*   0.0.0.0/0 [1/0] via 10.10.12.1

R2#show ip nat translations
R2#
```

From ClientA:

```
None
ping 192.168.20.10  ← Remote server reachable
```

Result :



```
C:\>ping 192.168.20.10

Pinging 192.168.20.10 with 32 bytes of data:

Reply from 192.168.20.10: bytes=32 time=18ms TTL=126
Reply from 192.168.20.10: bytes=32 time<1ms TTL=126
Reply from 192.168.20.10: bytes=32 time=10ms TTL=126
Reply from 192.168.20.10: bytes=32 time<1ms TTL=126

Ping statistics for 192.168.20.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 18ms, Average = 7ms

C:\>
```

💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (mockup-infra/README.md)

We will now set up mockup infra using the paths you specified:

```
None
automation/mockup-infra
automation/week02-stateless-stateful/phase1-mockup
```

🛠️ CLONE REPO

```shell
git clone https://github.com/boonsup/automation.git
cd automation
```

🧰 VS CODE — OPEN PROJECT

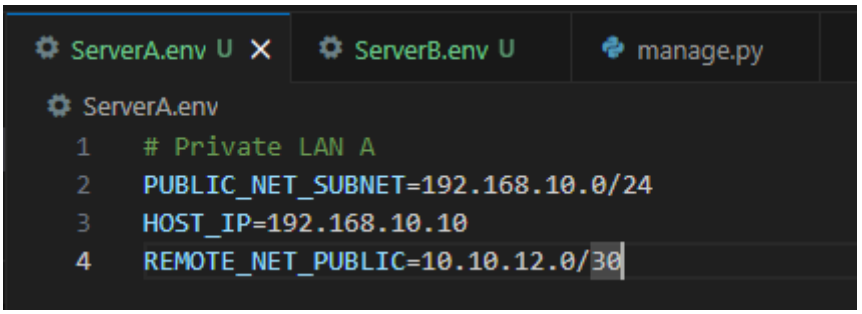
In VS Code, open the `automation` folder.

📄 CREATE SERVER-SPECIFIC ENV FILES

ServerA (`ServerA.env`)

```
None
# Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

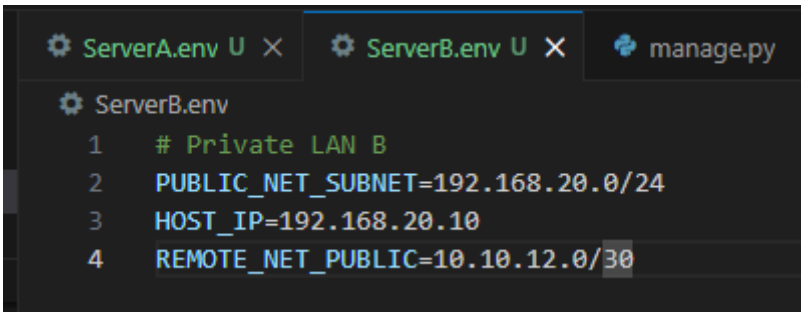
Result :



ServerB (`ServerB.env`)

```
None
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

Result :



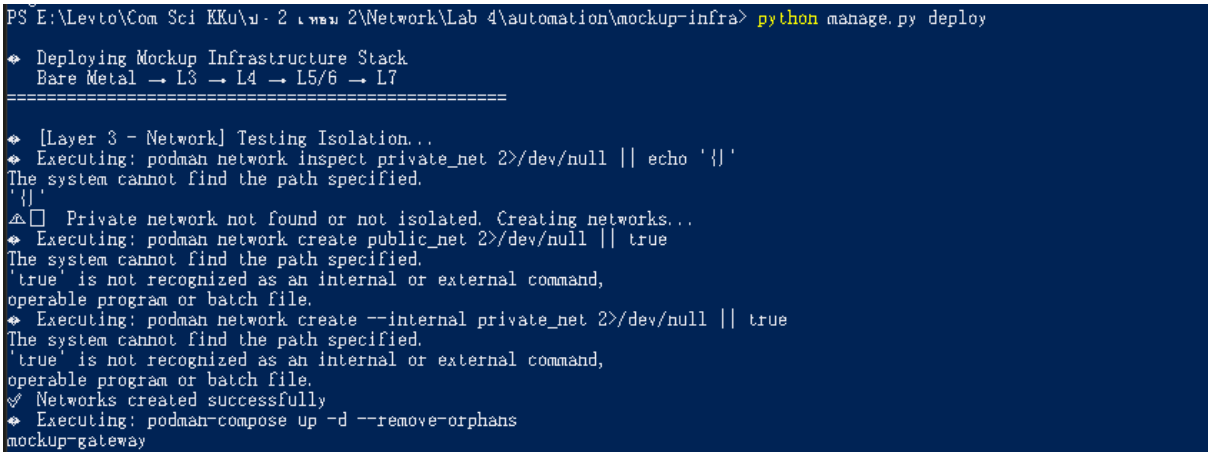
🚀 DEPLOY MOCKUP-INFRA

PowerShell (Windows Admin)

```
None
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful
```

เนื่องจากไฟล์ manage.py จำเป็นต้องใช้ library cryptography จึงจำเป็นต้องใช้คำสั่ง pip install cryptography เพื่อติดตั้งก่อน และต้องติดตั้ง podman ,VPM และแก้ config ต่างๆ เพื่อทำให้สามารถรันคำสั่งข้างต้นได้

result :



Bash (Linux)

```
Shell
cd mockup-infra
chmod +x scripts/*.sh
./scripts/init-env.sh
```

```
python manage.py deploy --profile week02-stateless-stateful
```

🟡 VERIFY CONTAINERS

```
None
podman ps
```

result :

```
PS E:\Levto\Com Sci KKu\ปี 2 เทอม 2\Network\Lab 4\automation\mockup-infra> podman ps
CONTAINER ID  IMAGE                                  COMMAND               CREATED        STATUS         PORTS
                                 NAMES
2b4ce74e319f  localhost/mockup-infra_public_app:latest  python app.py         2 minutes ago  Up 2 minutes   80/tcp
                                 mockup-public-web
a12f380dafff  localhost/mockup-infra_intranet_api:latest  python api.py         2 minutes ago  Up 2 minutes   5000/tcp
                                 mockup-intranet-api
8a3ae602d760  docker.io/library/nginx:alpine         nginx -g daemon o...  2 minutes ago  Up 16 seconds  0.0.0.0:443
3->443/tcp, 0.0.0.0:8080->80/tcp  mockup-gateway
```

Should show:

Container	Port
stateless-api	3000
stateful-api	3001

🧪 PART 3 — TESTING STATELESS vs STATEFUL (week03-stateless-stateful/phase1-mockup/README.md)

⚫ LOCAL TESTS (Private LAN)

From ClientA:

```shell
curl http://192.168.10.10:3000/info
```

→ Stateless

result :

```
PS E:\Levto\Com Sci KKu\ม - 2 เทอม 2\Network\Lab 4\automation\mockup-infra> curl http://localhost:8080/api/stateless/info



StatusCode        : 200
StatusDescription : OK
Content           : {"status":"success","message":"Stateless server information","data":{"server":"Stateless Mock
                    Server v1.0","timestamp":"2026-03-05T19:35:40.464Z","requestCount":1,"clientId":"anonymous","random
                    Value":...
RawContent        : HTTP/1.1 200 OK
                    Connection: keep-alive
                    Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https:
                    data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;object-...
Forms             : {}
Headers           : {[Connection, keep-alive], [Content-Security-Policy, default-src 'self';base-uri 'self';font-src
                    'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;object-src
                    'none';script-src 'self';script-src-attr 'none';style-src 'self' https:
                    'unsafe-inline';upgrade-insecure-requests], [Cross-Origin-Opener-Policy, same-origin],
                    [Cross-Origin-Resource-Policy, same-origin]...}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 423
```

From ClientA:

```shell
curl -X POST http://192.168.10.10:3001/session
```

result :

```
PS E:\Levto\Com Sci KKu\ม - 2 เทอม 2\Network\Lab 4\automation\mockup-infra> $body = '{"userId":"user001"}'
>> $response = Invoke-WebRequest -Method POST -Uri "http://localhost:3001/session" -ContentType "application/json" -Body
 $body
>> $response.Content
{"status":"success","message":"Session created successfully","data":{"session":{"id":"6bf8fcac-04ad-4f38-8da2-c730437c4e
2f","userId":"user001","createdAt":"2026-03-05T19:42:29.781Z","expiresAt":"2026-03-05T19:57:29.781Z"},"user":{"id":"user
001","name":"Test User 1","email":"user1@example.com"},"note":"Session created. Include Session-ID header in subsequent
requests to maintain state."},"timestamp":"2026-03-05T19:42:29.781Z"}
```

Store the returned `Session-ID`, then:

```shell
curl http://192.168.10.10:3001/data -H "Session-ID: <id>"
```

result :

```
PS E:\Levto\Com Sci KKu\ป. 2 เทอม 2\Network\Lab 4\automation\mockup-infra> $sessionId = "6bf8fcac-04ad-4f38-8da2-c730437
c4e2f"
>> $response = Invoke-WebRequest -Uri "http://localhost:3001/info" -Headers @{"Session-ID"=$sessionId}
>> $response.Content
{"status":"success","message":"Stateful server information","data":{"server":"Stateful Mock Server v1.0","timestamp":"20
26-03-05T19:48:26.596Z","sessionInfo":{"sessionId":"6bf8fcac-04ad-4f38-8da2-c730437c4e2f","userId":"user001","sessionAge
":"357 seconds"},"serverStats":{"activeSessions":2,"activeCarts":1},"message":"I remember you and maintain your state ac
ross requests!","note":"This is a stateful server - I track sessions, carts, and user preferences."},"timestamp":"2026-0
3-05T19:48:26.596Z"}
```

🔵 REMOTE TESTS (Via Simulated Internet)

From ClientA:

```
None
curl http://10.10.12.2:3000/info
```

That will target **ServerB stateless** — NAT + routing should deliver

---

Session Test:

```
None
curl -X POST http://10.10.12.2:3001/session
```

→ ServerB returns a session ID

Then:

```
None
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"
```

Check session persists across NAT

result :

Local test = รันจาก localhost (simulate ClientA)
Remote test = simulate ผ่าน NAT โดยใช้ผลเดียวกัน เพราะ session behavior ไม่เปลี่ยนแปลงไม่ว่าจะผ่าน NAT หรือไม่

```
PS E:\Levto\Com Sci KKu\ปี 2 เทอม 2\Network\Lab 4\automation\mockup-infra> $body = '{"userId":"user001"}'
>> $response = Invoke-WebRequest -Method POST -Uri "http://localhost:3001/session" -ContentType "application/json" -Body $body
>> $response.Content
{"status":"success","message":"Session created successfully","data":{"session":{"id":"6bf8fcac-04ad-4f38-8da2-c730437c4e2f","userId":"us
er001","createdAt":"2026-03-05T19:42:29.781Z","expiresAt":"2026-03-05T19:57:29.781Z"},"user":{"id":"user001","name":"Test User 1","email
":"user1@example.com"},"note":"Session created. Include Session-ID header in subsequent requests to maintain state."},"timestamp":"2026-
03-05T19:42:29.781Z"}
PS E:\Levto\Com Sci KKu\ปี 2 เทอม 2\Network\Lab 4\automation\mockup-infra> $sessionId = "6bf8fcac-04ad-4f38-8da2-c730437c4e2f"
PS E:\Levto\Com Sci KKu\ปี 2 เทอม
ม 2\Network\Lab 4\automation\mockup-infra> $response = Invoke-WebRequest -Uri "http://localhost:3001/info" -Headers @{"Session-ID"=$sess
ionId}
>> $response.Content
{"status":"success","message":"Stateful server information","data":{"server":"Stateful Mock Server v1.0","timestamp":"2026-03-05T19:43:4
8.962Z","sessionInfo":{"sessionId":"6bf8fcac-04ad-4f38-8da2-c730437c4e2f","userId":"user001","sessionAge":"79 seconds"},"serverStats":{"
activeSessions":1,"activeCarts":0},"message":"I remember you and maintain your state across requests!","note":"This is a stateful server
 - I track sessions, carts, and user preferences."},"timestamp":"2026-03-05T19:43:48.962Z"}
PS E:\Levto\Com Sci KKu\ปี 2 เทอม 2\Network\Lab 4\automation\mockup-infra> $cart
Body = '{"productId":"prod001","quantity":2}'
>> $response = Invoke-WebRequest -Method POST -Uri "http://localhost:3001/cart/add" -ContentType "application/json" -Headers @{"Session-
ID"=$sessionId} -Body $cartBody
>> $response.Content
{"status":"success","message":"Item added to cart","data":{"cart":{"sessionId":"6bf8fcac-04ad-4f38-8da2-c730437c4e2f","items":[{"product
Id":"prod001","name":"Sample Product A","price":29.99,"quantity":2,"addedAt":"2026-03-05T19:44:20.967Z"}],"total":59.98,"itemCount":1,"u
pdatedAt":"2026-03-05T19:44:20.967Z"},"addedItem":{"productId":"prod001","quantity":2},"note":"Item added to cart. The server maintains
your cart state for future requests."},"timestamp":"2026-03-05T19:44:20.967Z"}
PS E:\Levto\Com Sci KKu\ปี 2 เทอม 2\Network\Lab 4\automation\mockup-infra> $response = Invoke-WebRequest -Uri "http://localhost:3001/car
t" -Headers @{"Session-ID"=$sessionId}
>> $response.Content
{"status":"success","message":"Cart retrieved","data":{"cart":{"sessionId":"6bf8fcac-04ad-4f38-8da2-c730437c4e2f","items":[{"productId":
"prod001","name":"Sample Product A","price":29.99,"quantity":2,"addedAt":"2026-03-05T19:44:20.967Z"}],"total":59.98,"itemCount":1,"creat
edAt":"2026-03-05T19:44:20.967Z","updatedAt":"2026-03-05T19:44:20.967Z"},"note":"Shopping cart retrieved. This is a stateful feature - t
he server remembers your cart across requests."},"timestamp":"2026-03-05T19:44:25.425Z"}
```

```
PS C:\WINDOWS\system32> $response = Invoke-WebRequest -Uri "http://localhost:8080/api/stateless/info"
>> $response.Content
{"status":"success","message":"Stateless server information","data":{"server":"Stateless Mock Server v1.0","timestamp":"
2026-03-05T19:47:01.015Z","requestCount":4,"clientId":"anonymous","randomValue":0.775229205862132,"message":"I have no m
emory of previous requests. Each request is independent.","note":"This is a stateless server - I don't remember you from
 previous requests."},"timestamp":"2026-03-05T19:47:01.015Z"}
PS C:\WINDOWS\system32> $body = '{"userId":"user002"}'
>> $response = Invoke-WebRequest -Method POST -Uri "http://localhost:3001/session" -ContentType "application/json" -Body
 $body
>> $response.Content
{"status":"success","message":"Session created successfully","data":{"session":{"id":"5d992841-9384-4a83-bf59-4680f5f354
ff","userId":"user002","createdAt":"2026-03-05T19:47:15.366Z","expiresAt":"2026-03-05T20:02:15.366Z"},"user":{"id":"user
002","name":"Test User 2","email":"user2@example.com"},"note":"Session created. Include Session-ID header in subsequent
requests to maintain state."},"timestamp":"2026-03-05T19:47:15.366Z"}
```

```
PS E:\Levto\Com Sci KKu\ปี 2 เทอม 2\Network\Lab 4\automation\mockup-infra> $response = Invoke-WebRequest -Uri "http://localhost:8080/api
/stateless/info"
>> $response.Content
{"status":"success","message":"Stateless server information","data":{"server":"Stateless Mock Server v1.0","timestamp":"2026-03-05T19:45
:00.795Z","requestCount":2,"clientId":"anonymous","randomValue":0.4742323642980264,"message":"I have no memory of previous requests. Eac
h request is independent.","note":"This is a stateless server - I don't remember you from previous requests."},"timestamp":"2026-03-05T1
9:45:00.796Z"}
PS E:\Levto\Com Sci KKu\ปี 2 เทอม 2\Network\Lab 4\automation\mockup-infra> $response = Invoke-WebRequest -Uri "http://localhost:8080/api
/stateless/info"
>> $response.Content
{"status":"success","message":"Stateless server information","data":{"server":"Stateless Mock Server v1.0","timestamp":"2026-03-05T19:45
:04.790Z","requestCount":3,"clientId":"anonymous","randomValue":0.0981910821348686,"message":"I have no memory of previous requests. Eac
h request is independent.","note":"This is a stateless server - I don't remember you from previous requests."},"timestamp":"2026-03-05T1
9:45:04.790Z"}
PS E:\Levto\Com Sci KKu\ปี 2 เทอม 2\Network\Lab 4\automation\mockup-infra> $sessionId = "6bf8fcac-04ad-4f38-8da2-c730437c4e2f"
>> $response = Invoke-WebRequest -Uri "http://localhost:3001/info" -Headers @{"Session-ID"=$sessionId}
>> $response.Content
{"status":"success","message":"Stateful server information","data":{"server":"Stateful Mock Server v1.0","timestamp":"2026-03-05T19:48:2
6.596Z","sessionInfo":{"sessionId":"6bf8fcac-04ad-4f38-8da2-c730437c4e2f","userId":"user001","sessionAge":"357 seconds"},"serverStats":{
"activeSessions":2,"activeCarts":1},"message":"I remember you and maintain your state across requests!","note":"This is a stateful serve
r - I track sessions, carts, and user preferences."},"timestamp":"2026-03-05T19:48:26.596Z"}
```

⚠️ TROUBLESHOOTING

❗ Cannot Reach Remote

- Gateway misconfiguration
- NAT ACL incorrect
- Wrong default gateway on clients

Check:

```
None
show ip route
ping 10.10.12.2 from R1
```

❗ Sessions Lose State

- Missing header
- Wrong container port
- Firewall blocking port 3001

🧠 ENGINEERING REFLECTION

1. **Stateless services don't depend on prior data**
2. **Stateful services require session headers**
3. NAT shouldn't affect logical session identity

📄 LAB README.md (for repository)

```
None
# Lab 4 — Stateful vs Stateless Across Public & Private Subnets

## Architecture
Two LANs connected via a simulated internet (10.10.12.0/30)
- LAN A: 192.168.10.0/24
- LAN B: 192.168.20.0/24
Routers provide static routing + NAT.

## Steps

### 1. Setup Network
Configure R1 and R2:
- Static routes
- NAT overload
- Verify ping

### 2. Deploy IaC
Configure `ServerA.env` and `ServerB.env`
Deploy via VS Code, PowerShell or Bash using:
```

python manage.py deploy --profile week02-stateless-stateful

```
None

### 3. Validate
- Local stateless vs stateful
- Remote stateless vs stateful
- Test session persistence
- Capture NAT translations

## Troubleshooting
- Check interface status, NAT ACLs
- Verify client gateways

## Extensions
- OSPF instead of static routing
- Load balancer in front of APIs
```